

將自訂模型整合至應用程式

來源：<https://codelabs.developers.google.com/tflite-computer-vision-update-app?hl=zh-tw>

步驟數：10

在這個程式碼研究室中，您將瞭解如何使用 ML Kit 自訂模型，將自訂圖片分類模型整合至 Android 或 iOS 應用程式。

目錄

1. [事前準備](#)
2. [取得範例應用程式](#)
3. [更新 build.gradle，使用自訂 ML Kit 模型](#)
4. [新增 TFLite 模型](#)
5. [更新自訂模型的程式碼](#)
6. [取得 iOS Start 應用程式](#)
7. [使用自訂 ML Kit 圖像標籤 Pod](#)
8. [將 TFLite 模型新增至 Xcode](#)
9. [更新自訂模型的程式碼](#)
10. [恭喜！](#)

1. 事前準備

在本系列的第一個程式碼研究室中，您建立了一個非常簡單的應用程式，該應用程式使用圖像標籤功能剖析圖像內容。你傳送了雛菊的圖片，而模型回覆的內容是花瓣或天空等。接著，在第二個程式碼研究室中，您改用 Python 訓練新的自訂模型，辨識五種不同類型的花朵。

在本程式碼研究室中，您將使用第二個實驗室的模型，更新第一個實驗室的應用程式！

如要取得本程式碼研究室的完整原始碼，請複製[這個](#)存放區。您會看到 Android 和 iOS 的子目錄。如果您想繼續操作，可以將先前的程式碼研究室程式碼做為 ImageClassifierStep1。本程式碼研究室完成後的程式碼為 ImageClassifierStep2。

如果是 iOS 裝置，請跳到步驟 6！

必要條件

- 您應已完成本學習路徑的前兩個程式碼研究室

建構項目和學習內容

- 將在上一個實驗室中訓練的自訂模型整合至 Android 或 iOS 應用程式

軟硬體需求

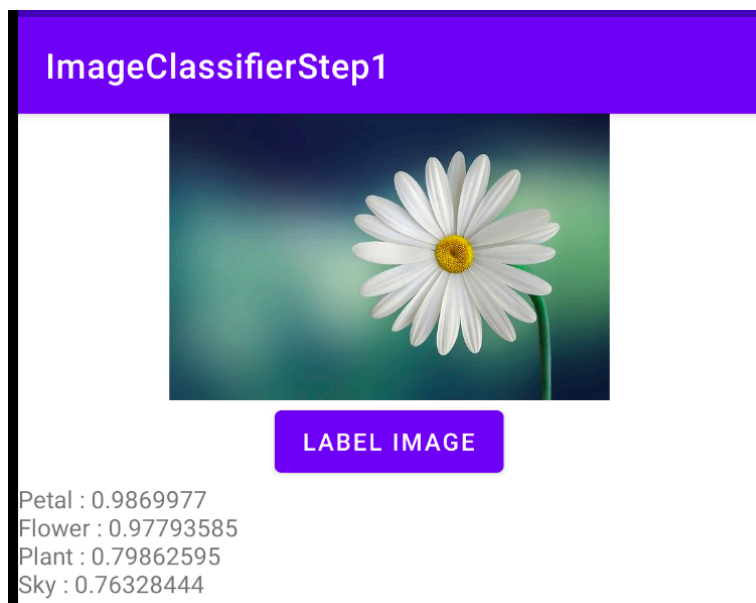
- Android Studio (可從 developer.android.com/studio 取得)，適用於實驗室的 Android 部分
 - Xcode (可透過 Apple App Store 下載)，適用於實驗室的 iOS 部分
-

步驟 2 · 約 2 分鐘

2. 取得範例應用程式

首先，您需要「在 Android 或 iOS 上建構第一個 Computer Vision 應用程式」程式碼研究室中的應用程式。如果您已完成實驗室，則會稱為 ImageClassifierStep1。如果不想完成實驗室，可以從 [repo](#) 複製完成的版本

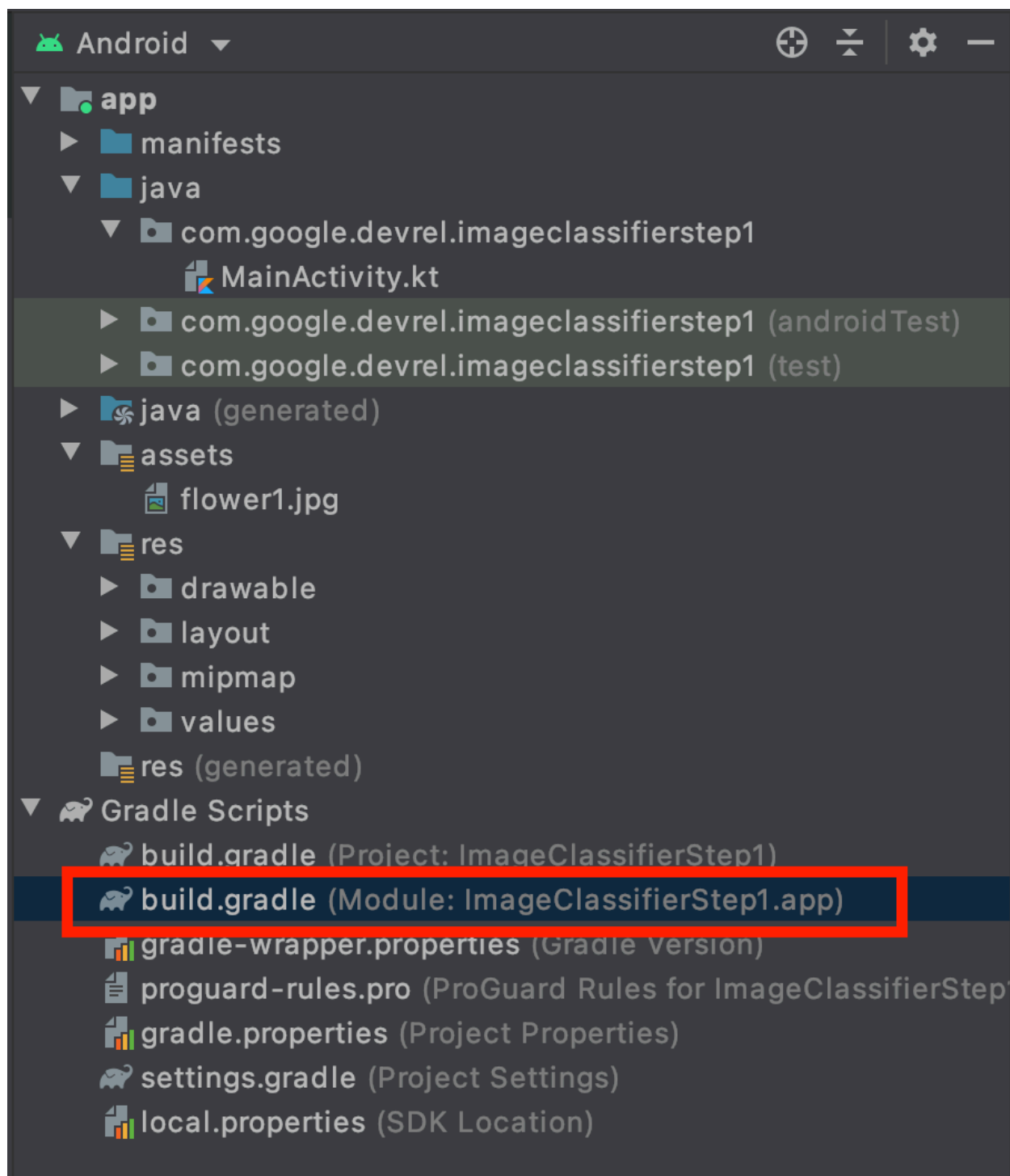
在 Android Studio 中開啟，視需要更新，然後執行應用程式，確認是否正常運作。畫面應如下所示：



這個應用程式相當原始，但只用少許程式碼就展現了強大的功能。不過，如果您希望系統將這朵花辨識為雛菊，而不只是花朵，就必須更新應用程式，使用「為圖片分類器建立自訂模型」程式碼研究室中的自訂模型。

3. 更新 build.gradle，使用自訂 ML Kit 模型

1. 使用 Android Studio 找出應用程式層級的 `build.gradle` 檔案。最簡單的方法是在專案瀏覽器中執行這項操作。確認頂端已選取「Android」，底部會顯示「Gradle Scripts」資料夾。
2. 開啟模組專用的檔案，應用程式名稱後方會加上「.app」，如下所示 (模組：ImageClassifierStep1.app)：



3. 在檔案底部找到「dependencies」設定。您應該會看到下面這行：

```
implementation 'com.google.mlkit:image-labeling:17.0.1'
```

版本號碼可能有所不同。請前往 ML Kit 網站查看最新版本號碼：<https://developers.google.com/ml-kit/vision/image-labeling/android>

4. 請將此項替換為自訂圖片標籤庫參照。如要查看版本號碼，請前往：<https://developers.google.com/ml-kit/vision/image-labeling/custom-models/android>

```
implementation 'com.google.mlkit:image-labeling-custom:16.3.1'
```

5. 此外，您還會新增在上一個實驗室中建立的 .tflite 模型。您不希望 Android Studio 編譯應用程式時壓縮這個模型，因此請務必在相同 `build.gradle` 檔案的 **Android** 區段中使用這項設定：

```
aaptOptions{
    noCompress "tflite"
}
```

請確認該設定不在任何其他設定中。應直接以巢狀形式列於 `android` 標記底下。範例如下：

```
android {
    compileSdkVersion 30
    buildToolsVersion "30.0.3"
    aaptOptions{
        noCompress "tflite"
    }
    defaultConfig {
        applicationId "com.google.devrel.imageclassifierstep2"
        minSdkVersion 23
        targetSdkVersion 30
        versionCode 1
        versionName "1.0"

        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
    }

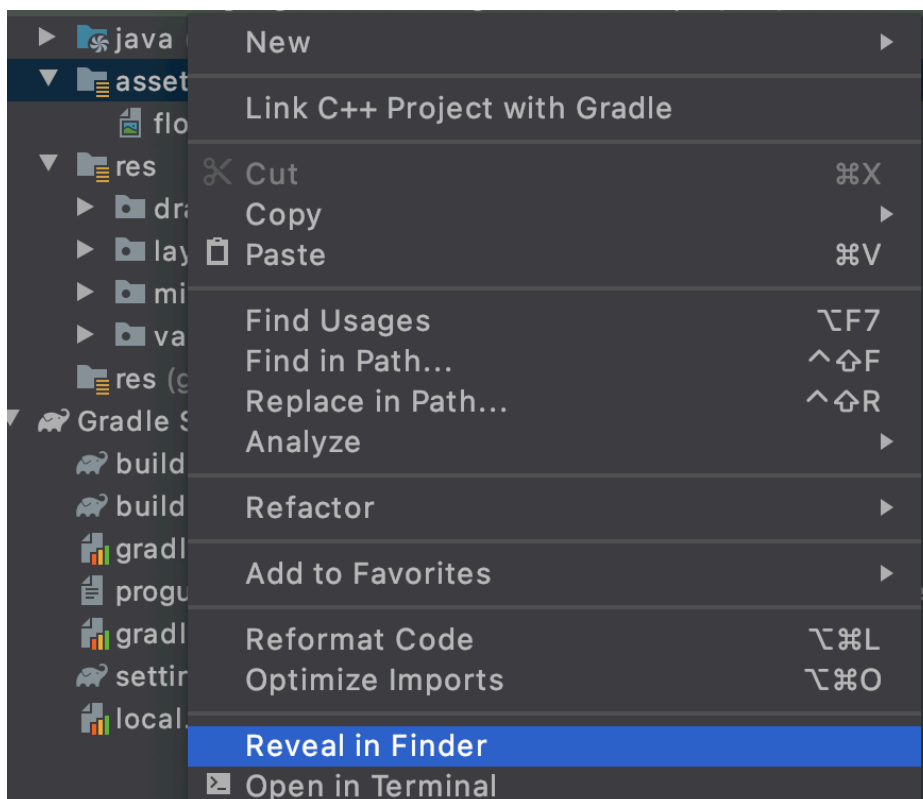
    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'
        }
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_1_8
        targetCompatibility JavaVersion.VERSION_1_8
    }
    kotlinOptions {
        jvmTarget = '1.8'
    }
}
```

4. 新增 TFLite 模型

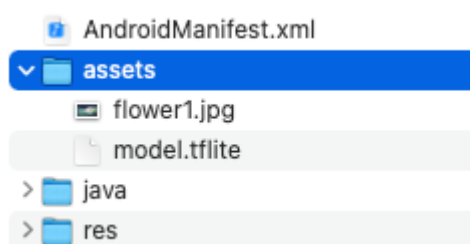
在上一個程式碼研究室中，您建立了自訂模型，並將其下載為 `model.tflite`。

在專案中，找出目前包含 `flower1.jpg` 的 **assets** 資料夾。將模型複製到該資料夾，方法如下：

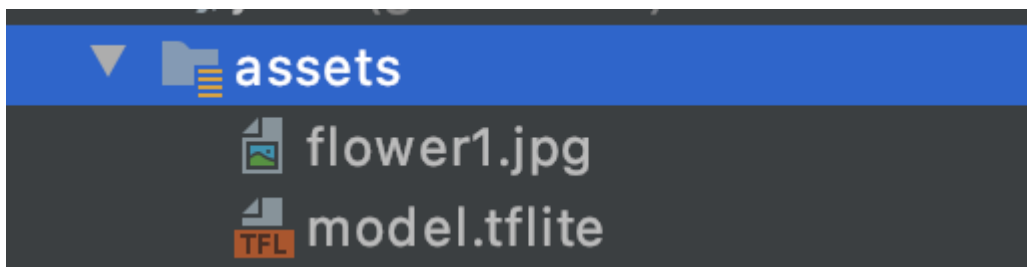
1. 在 Android Studio 中，對「Assets」資料夾按一下滑鼠右鍵。在開啟的選單中選取「在 Finder 中顯示」。(在 Windows 上為「在檔案總管中顯示」，在 Linux 上為「在檔案中顯示」)。



2. 系統會將您帶往檔案系統中的目錄。將 `model.tflite` 檔案複製到該目錄中，與 `flower1.jpg` 放在一起



Android Studio 會更新，顯示資產資料夾中的兩個檔案：



您現在可以更新程式碼了。

5. 更新自訂模型的程式碼

第一步是新增一些程式碼，載入自訂模型。

1. 在 `MainActivity` 檔案中，將下列內容新增至 `onCreate`，緊接在 `setContentView(R.layout.activity_main)` 這一行下方。

這會使用 `LocalModel` 從 `model.tflite` 資產建構。如果 Android Studio 將「`LocalModel`」標示為紅色，請按 **Alt + Enter** 鍵匯入程式庫。系統應會為您新增 `com.google.mlkit.common.model.LocalModel` 的匯入項目。

```
val localModel = LocalModel.Builder()
    .setAssetFilePath("model.tflite")
    .build()
```

先前在 `btn.setOnClickListener` 處理常式中，您使用的是預設模型。設定時使用的代碼：

```
val labeler = ImageLabeling.getClient(ImageLabelerOptions.DEFAULT_OPTIONS)
```

您將替換該模型，改用自訂模型。

2. 設定自訂選項物件：

```
val options = CustomImageLabelerOptions.Builder(localModel)
    .setConfidenceThreshold(0.7f)
    .setMaxResultCount(5)
    .build()
```

這會以自訂選項取代預設選項。可信度門檻會設定預測結果品質的標準，回顧本程式碼研究室頂端的範例，圖片是雛菊，您會看到 4 項預測結果，每項旁邊都有值，例如「Sky」是 0.7632。

使用高可信度門檻，即可有效過濾掉品質較低的結果。舉例來說，如果將這個值設為 0.9，系統就不會傳回優先順序低於該值的任何標籤。`setMaxResultCount()` 在類別眾多的模型中很有用，但這個模型只有 5 個類別，因此請將其保留為 5。

現在您已取得標籤器選項，可以將標籤器的例項變更為：

```
val labeler = ImageLabeling.getClient(options)
```

其餘程式碼會照常執行。讓我們來試試看！

ImageClassifierStep2



LABEL IMAGE

daisy : 0.9588599

您可以看到，這朵花現在被辨識為雛菊，機率為 0.959！

假設您新增了第二張花朵圖片，並使用該圖片重新執行：

ImageClassifierStep2



LABEL IMAGE

roses : 0.93792176

並辨識出這是玫瑰花。

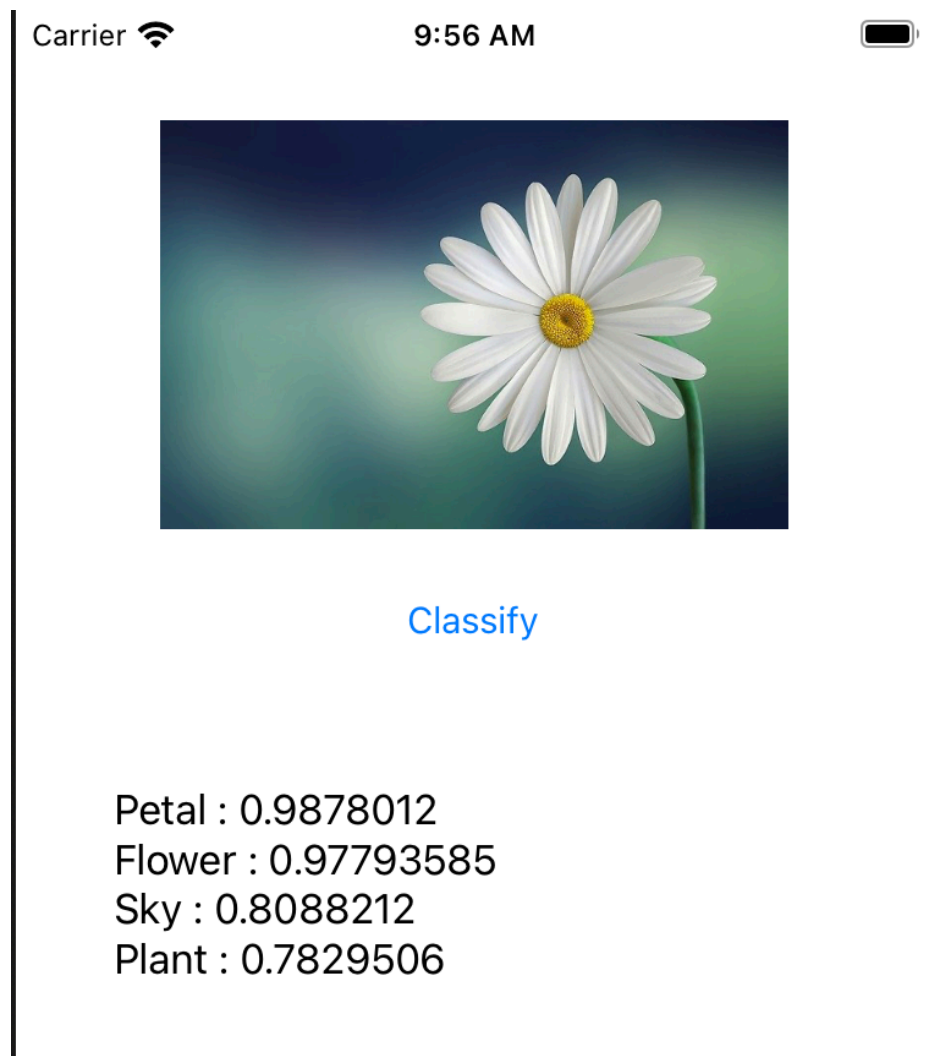
您可能會好奇，為什麼是「roses」而不是「rose」。這是因為資料集中的標籤是由資料夾名稱提供，但很遺憾的是，這些資料夾名稱不太一致，有時使用單數 (例如「daisy」)，有時使用複數 (例如「roses」)。請勿將這項功能與嘗試計算圖片中項目數量的模型混淆，這項功能比該模型原始許多，只能辨識花朵類型！

6. 取得 iOS Start 應用程式

1. 首先，您需要第一個程式碼研究室的應用程式。如果您已完成實驗室，則會稱為 ImageClassifierStep1。如果不想完成實驗室，可以從 [repo](#) 複製完成的版本。請注意，存放區中沒有 Pod 和 .xcworkspace，因此請務必從與 .xcproject 相同的目錄執行「pod install」，再繼續下一個步驟。
2. 在 Xcode 中開啟 ImageClassifierStep1.xcworkspace。請注意，您應使用 .xcworkspace，而非 .xcproject，因為您已使用 Pod 組合 ML Kit，而工作區會載入這些 Pod。

在本實驗室的其餘部分，我會在 iPhone 模擬器中執行應用程式，該模擬器應支援程式碼研究室的建構目標。如要使用自己的裝置，可能需要在專案設定中變更建構目標，以符合 iOS 版本。

執行資訊方塊，畫面應如下所示：



請注意，這些分類非常籠統，例如花瓣、花朵、天空。您在上一個程式碼研究室中建立的模型經過訓練，可偵測 5 種花卉，包括雛菊。

在本程式碼研究室的其餘部分，您將瞭解如何使用自訂模型升級應用程式。

步驟 7 · 約 4 分鐘

7. 使用自訂 ML Kit 圖像標籤 Pod

第一個應用程式使用 Pod 檔案取得基本的 ML Kit 圖片標籤器程式庫和模型。您需要更新該項目，才能使用自訂圖片標籤程式庫。

1. 在專案目錄中找出名為 `podfile` 的檔案。開啟後，畫面應如下所示：

```
platform :ios, '10.0'

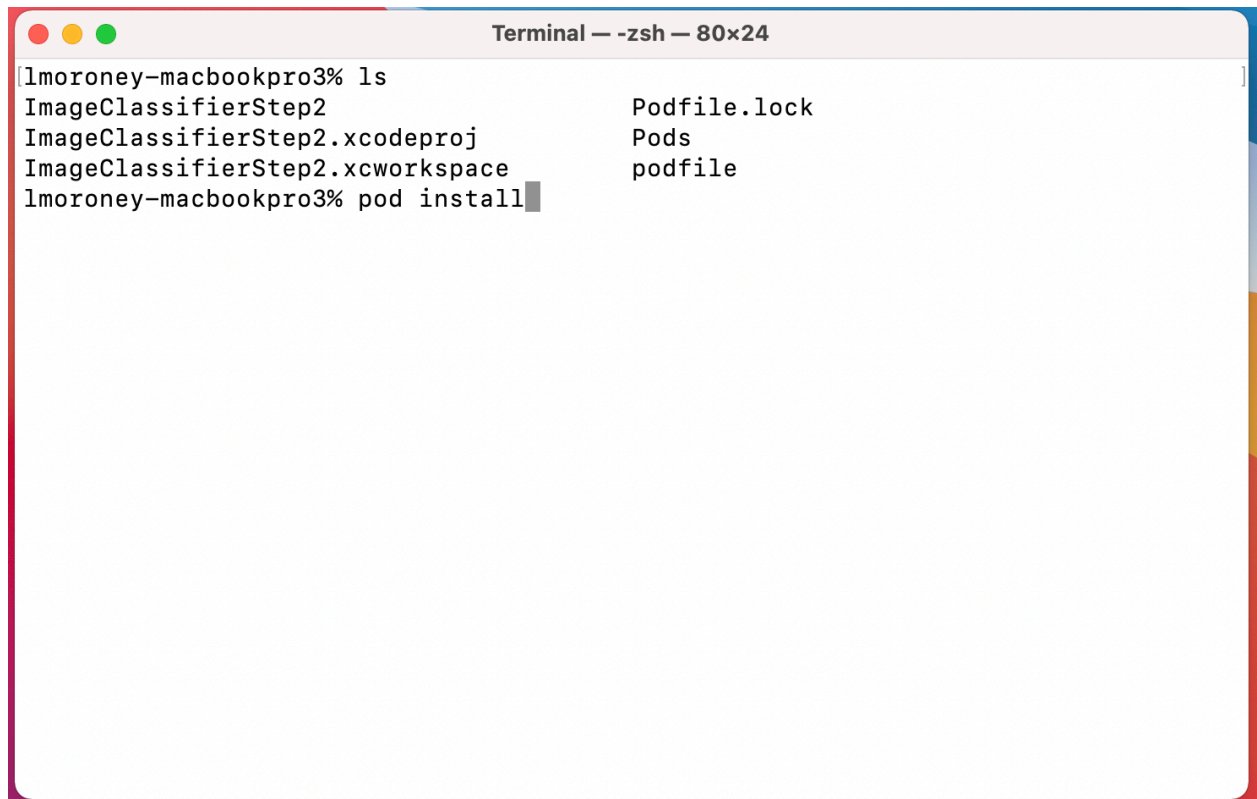
target 'ImageClassifierStep1' do
  pod 'GoogleMLKit/ImageLabeling'
end
```

2. 將 Pod 宣告從 `ImageLabeling` 變更為 `ImageLabelingCustom`，如下所示：

```
platform :ios, '10.0'

target 'ImageClassifierStep1' do
  pod 'GoogleMLKit/ImageLabelingCustom'
end
```

3. 完成後，請使用終端機前往含有 Podfile (以及 .xcworkspace) 的目錄，然後執行 `pod install`。



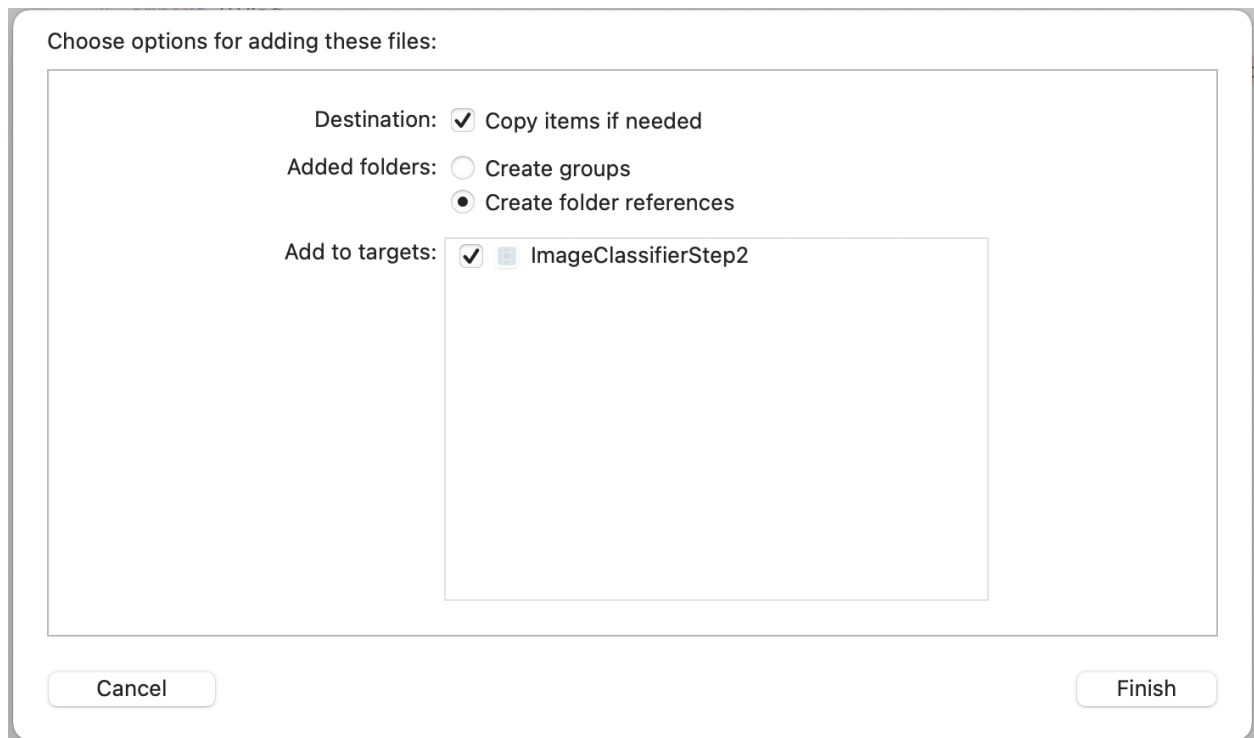
稍待片刻，系統就會移除 MLKitImageLabeling 程式庫，並新增自訂程式庫。現在可以開啟 .xcworkspace 編輯程式碼。

8. 將 TFLite 模型新增至 Xcode

在先前的程式碼研究室中，您建立了自訂模型，並將其下載為 `model.tflite`。如果沒有，請返回並執行該程式碼研究室，或前往[這裡](#)查看 Colab 程式碼。如果無法存取 Google Colab，請點選[這個](#)連結開啟筆記本

1. 在 Xcode 中開啟工作區，然後將 `model.tflite` 拖曳至專案。這個檔案應與其他檔案 (例如 `ViewController.swift` 或 `Main.storyboard`) 位於同一資料夾。
2. 畫面上會彈出對話方塊，其中含有新增檔案的選項。請務必選取「新增至目標」，否則模型部署到裝置時，不會與應用程式一併部署。

請注意，如果您從「Add to Targets」項目開始，並逐步完成本實驗室，則該項目會顯示「ImageClassifierStep1」；如果您直接跳到完成的程式碼，則會顯示「ImageClassifierStep2」(如圖所示)。



確保您可以載入模型。我們會在下一個步驟中說明如何進行這項操作。

9. 更新自訂模型的程式碼

1. 開啟 `ViewController.swift` 檔案。您可能會在檔案頂端的「`import MLKitImageLabeling`」看到錯誤。這是因為您更新 Pod 檔案時，移除了通用圖片標籤程式庫。請隨意刪除這行，並更新為下列內容：

```
import MLKitVision
import MLKit
import MLKitImageLabelingCommon
import MLKitImageLabelingCustom
```

您可能很快就讀完這些程式碼，並認為它們重複了相同的程式碼！但最後是「Common」和「Custom」！

2. 接著，您將載入在上一個步驟中新增的自訂模型。找出 `getLabels()` 函式。在 `visionImage.orientation = image.imageOrientation` 這一行下方新增以下幾行：

```
// Add this code to use a custom model
let localModelFilePath = Bundle.main.path(forResource: "model", ofType: "tflite")
let localModel = LocalModel(path: localModelFilePath!)
```

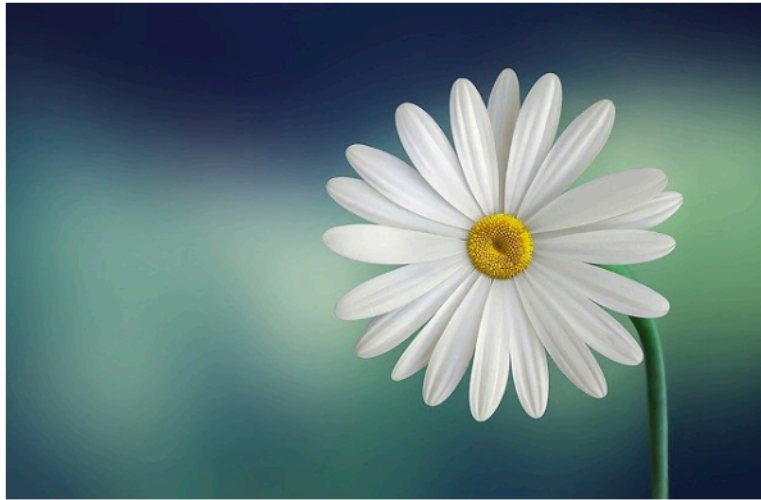
3. 找出指定通用 `ImageLabeler` 選項的程式碼。由於這些程式庫已移除，系統可能會顯示錯誤訊息：

```
let options = ImageLabelerOptions()
```

請將該行內容替換為下列程式碼，使用 `CustomImageLabelerOptions` 並指定本機模型：

```
let options = CustomImageLabelerOptions(localModel: localModel)
```

就是這麼簡單！現在請嘗試執行應用程式！嘗試分類圖片時，模型應該會更準確，並告訴你這張圖片很可能是雛菊！



Classify

daisy : 0.9503365

假設您新增了第二張花朵圖片，並重新執行：



Classify

roses : 0.91124976

應用程式成功偵測到這張圖片符合「玫瑰」標籤！

步驟 10 · 約 1 分鐘

10. 恭喜！

您現在已從建構使用通用模型辨識圖片內容的應用程式，進展到建立自己的機器學習模型來辨識特定事物 (例如花卉)，然後更新應用程式以使用自訂模型。

當然，由於這類應用程式依賴的是隨附的圖片素材資源，因此功能非常有限。不過，機器學習部分運作良好。舉例來說，您可以使用 `AndroidX Camera` 從現行動態饋給擷取影格，並進行分類，看看手機辨識出哪些花朵！

從這裡開始，您就能發揮無限創意。如果您有自己的資料 (不限於花卉)，就能運用電腦視覺技術，建構可辨識這些資料的應用程式。這些只是進入更廣闊世界的第一步，希望您喜歡這些練習！
